MONTH 1: CORE JAVA AND OBJECT-ORIENTED PROGRAMMING

• Week 1: Java Basics
Tasks:
Introduction to Java: Setup, basic syntax, and structure.
Understanding variables, data types, operators, and control flow (loops, conditionals).
Basic input/output in Java.
Project: Create a basic command-line calculator that supports operations like addition, subtraction, multiplication, and division.
• Week 2: Object-Oriented Programming (OOP)
Tasks:
Learn the principles of OOP: classes, objects, inheritance, polymorphism, encapsulation, and abstraction.
Understand constructors, methods, and access modifiers.
Project: Develop a simple banking application that allows users to create accounts, deposit/withdraw money, and view account details using OOP concepts.
• Week 3: Exception Handling and File I/O
Tasks:
Learn about exception handling in Java (try-catch, throw, throws).
Understand file handling in Java: reading from and writing to files.
Project: Extend the banking application to handle errors gracefully using exception handling. Add functionality to save account data to a file and load it upon startup.
• Week 4: Collections Framework
Tasks:
Introduction to the Java Collections Framework: List, Set, Map, and their implementations.
Understand iterators and how to manipulate data using collections.
Project: Create a student management system that stores student records (name, roll number, grades) using collections. Implement functionalities to add, remove, and search for students.

MONTH 2: ADVANCED JAVA CONCEPTS

Week 5: Generics and Multithreading
Tasks:
Learn about generics in Java and how they provide type safety.
Introduction to multithreading: threads, synchronization, and thread lifecycle.
Project: Develop a multi-threaded application that simulates a ticket booking system with concurrent users trying to book tickets.
Week 6: Java Networking
Tasks:
Learn about networking in Java: Sockets, ServerSocket, and how to create networked applications.
Implement basic client-server communication.
Project: Create a simple chat application where multiple clients can connect to a server and exchange messages.
Week 7: JDBC and Database Connectivity
Tasks:
Introduction to Java Database Connectivity (JDBC): connecting to a database, executing queries, and handling results.
Learn about CRUD operations using JDBC.
Project: Extend the student management system to connect to a database. Implement functionality to add, update, delete, and view student records from the database.
Week 8: Java GUI Development (Swing)
Tasks:
Learn the basics of building graphical user interfaces (GUIs) in Java using Swing.
Understand event handling and layout managers.
Project: Create a GUI-based version of the banking or student management system developed earlier. Add features like buttons, forms, and menus.

MONTH 3: FRAMEWORKS, APIS, AND FINAL PROJECT

Week 9: Introduction to Spring Framework
Tasks:
Introduction to the Spring Framework: Spring Boot, Dependency Injection, and Spring MVC.
Learn how to create a simple web application using Spring Boot.
Project: Develop a basic CRUD web application using Spring Boot, such as a simple blog platform where users can create, edit, and delete posts.
Week 10: RESTful Web Services
Tasks:
Learn about RESTful web services in Java using Spring Boot.
Understand how to create, test, and secure REST APIs.
Project: Create a RESTful API for the student management system, allowing external systems to interact with it.
Week 11: Unit Testing and Best Practices
Tasks:
Introduction to unit testing in Java using JUnit and Mockito.
Learn about best practices for writing clean, maintainable Java code.
Project: Write unit tests for the applications developed during the internship. Ensure that all code follows Java best practices and is well-documented.
Week 12: Final Project & Presentation
Project: Develop a comprehensive final project that incorporates all the concepts learned throughout the internship. This could be a complex application such as an e-commerce platform, a social networking site, or a task management system. The project should include:

A well-designed GUI.
Database integration.
RESTful web services.
Multithreading (if applicable).
Proper exception handling and unit testing.
Presentation: Prepare a detailed presentation of the final project, including a live demo, code walkthrough, and explanation of design decisions. Include documentation that outlines the project's functionality, architecture, and how to deploy it.